



NETAJI SUBHAS UNIVERSITY OF TECHNOLOGY

Sector 3, Dwarka, New Delhi - 110078

PRACTICAL FILE

Blockchain and Its Applications

Subject Code: MTITE104

Submitted To:

Department of Information
Technology
NSUT, New Delhi

Submitted By:

Name: Aniket Sonker
Roll No: 2026PIT4227
Branch: Information Technology

Contents

1	Assignment 1: Implementation of MD5 Algorithm	1
1.1	Theory & Overview	1
1.2	Python Code	1
1.3	Sample Output	1
1.4	Result & Conclusion	1
2	Assignment 2: Implementation of SHA-256 Algorithm	2
2.1	Theory & Algorithm	2
2.2	C++ Code	2
2.3	Sample Output	4
2.4	Result & Conclusion	4
3	Assignment 3: Implementation and Verification of RSA Digital Signature Scheme	5
3.1	Theory & Overview	5
3.2	Python Code	5
3.3	Sample Output	6
3.4	Result & Conclusion	6
4	Assignment 4: Implementation of ElGamal Digital Signature Scheme	7
4.1	Theory & Mathematical Formulation	7
4.2	Python Code	7
4.3	Sample Output	9
4.4	Observation & Result	9

1 Assignment 1: Implementation of MD5 Algorithm

Aim / Objective: Implement the MD5 hashing algorithm in Python/Java/C++.

1.1 Theory & Overview

MD5 (Message-Digest Algorithm 5) is a widely used cryptographic hash function that produces a 128-bit (16-byte) hash value, typically expressed as a 32-character hexadecimal number. It was designed by Ronald Rivest in 1991 to replace an earlier hash function, MD4.

1.2 Python Code

```
1 import hashlib
2
3 def get_md5(data: bytes | str) -> str:
4     if isinstance(data, str):
5         data = data.encode('utf-8')
6     return hashlib.md5(data).hexdigest()
7
8 if __name__ == "__main__":
9     text = "hello world"
10    print(f"MD5 digest: {get_md5(text)}")
```

Listing 1: MD5 Algorithm Implementation in Python

1.3 Sample Output

```
$ python assignment1.py
MD5 digest: 5eb63bbbe01eeed093cb22bb8f5acdc3
```

1.4 Result & Conclusion

The MD5 algorithm was successfully implemented in Python using standard libraries. The input string "hello world" produces the expected 128-bit digest:

5eb63bbbe01eeed093cb22bb8f5acdc3

2 Assignment 2: Implementation of SHA-256 Algorithm

Aim / Objective: Implement the SHA-256 Hashing Algorithm in C++.

2.1 Theory & Algorithm

SHA-256 is a cryptographic hash function belonging to the SHA-2 family, designed by the NSA. It takes an input message of arbitrary length and produces a deterministic 256-bit (32-byte) output, known as a message digest or hash signature.

Key Steps in SHA-256 Processing:

1. **Message Padding:** The message is padded so its total length in bits is congruent to 448 (mod 512). A single bit '1' (0x80) is appended, followed by zero bits (0x00). Finally, a 64-bit Big-Endian integer representing the original length in bits is appended to make the overall size a multiple of 512 bits (64 bytes).
2. **Initialization of State Variables:** Eight 32-bit working state registers (H_0 to H_7) are initialized using the fractional parts of the square roots of the first 8 prime numbers (2 to 19).
3. **Message Schedule Expansion:** Each 512-bit block is split into sixteen 32-bit words ($W_0 \dots W_{15}$). Words $W_{16} \dots W_{63}$ are computed using rotation and shift operators.
4. **Compression Function (64 Rounds):** Working variables undergo 64 mixing rounds utilizing round constants (cube roots of the first 64 prime numbers) and non-linear functions $\text{Ch}(e, f, g)$ and $\text{Maj}(a, b, c)$.
5. **Final Digest Output:** Intermediate hash blocks are updated and concatenated into a 64-character hexadecimal string representation.

2.2 C++ Code

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <sstream>
5 #include <iomanip>
6 #include <stdint>
7
8 class SHA256 {
9 private:
10     static uint32_t rotr(uint32_t x, uint32_t n) {
11         return (x >> n) | (x << (32 - n));
12     }
13     static const uint32_t K[64];
14
15 public:
16     static std::string hash(const std::string& input) {
17         // Step 1: Pre-processing (Padding)
18         std::vector<uint8_t> data(input.begin(), input.end());
19         uint64_t bit_len = data.size() * 8;
20
21         data.push_back(0x80); // Append '1' bit
22         while ((data.size() % 64) != 56) {
```

```

23     data.push_back(0x00);
24 }
25
26 // Append length as 64-bit integer (Big Endian)
27 for (int i = 7; i >= 0; --i) {
28     data.push_back(static_cast<uint8_t>((bit_len >> (i * 8)) & 0xFF));
29 }
30
31 // Step 2: Initialize hash values
32 uint32_t H[8] = {
33     0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a,
34     0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19
35 };
36
37 // Step 3: Process message in 512-bit chunks
38 for (size_t chunk = 0; chunk < data.size(); chunk += 64) {
39     uint32_t W[64];
40     for (int t = 0; t < 16; ++t) {
41         W[t] = (data[chunk + t * 4 + 0] << 24) |
42             (data[chunk + t * 4 + 1] << 16) |
43             (data[chunk + t * 4 + 2] << 8) |
44             (data[chunk + t * 4 + 3]);
45     }
46
47     for (int t = 16; t < 64; ++t) {
48         uint32_t s0 = rotr(W[t - 15], 7) ^ rotr(W[t - 15], 18) ^ (W[t - 15]
>> 3);
49         uint32_t s1 = rotr(W[t - 2], 17) ^ rotr(W[t - 2], 19) ^ (W[t - 2] >>
10);
50         W[t] = W[t - 16] + s0 + W[t - 7] + s1;
51     }
52
53     uint32_t a = H[0], b = H[1], c = H[2], d = H[3];
54     uint32_t e = H[4], f = H[5], g = H[6], h = H[7];
55
56     for (int t = 0; t < 64; ++t) {
57         uint32_t S1 = rotr(e, 6) ^ rotr(e, 11) ^ rotr(e, 25);
58         uint32_t ch = (e & f) ^ ((~e) & g);
59         uint32_t temp1 = h + S1 + ch + K[t] + W[t];
60         uint32_t S0 = rotr(a, 2) ^ rotr(a, 13) ^ rotr(a, 22);
61         uint32_t maj = (a & b) ^ (a & c) ^ (b & c);
62         uint32_t temp2 = S0 + maj;
63
64         h = g;
65         g = f;
66         f = e;
67         e = d + temp1;
68         d = c;
69         c = b;
70         b = a;
71         a = temp1 + temp2;
72     }
73
74     H[0] += a; H[1] += b; H[2] += c; H[3] += d;
75     H[4] += e; H[5] += f; H[6] += g; H[7] += h;
76 }
77
78 // Step 4: Final output format
79 std::ostringstream ss;
80 for (int i = 0; i < 8; ++i) {
81     ss << std::hex << std::setfill('0') << std::setw(8) << H[i];
82 }
83 return ss.str();
84 }
85 };
86
87 const uint32_t SHA256::K[64] = {
88     0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b, 0x59f111f1,
89     0x923f82a4, 0xab1c5ed5,
90     0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3, 0x72be5d74, 0x80deb1fe,
91     0x9bdc06a7, 0xc19bf174,
92     0xe49b69c1, 0xefbe4786, 0x0fc19de6, 0x240ca1cc, 0x2de32061, 0x4a7484aa,
93     0x5cb0a9dc, 0x76f988da,

```

```
91     0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147,
92     0x06ca6351, 0x14292967,
93     0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13, 0x650a7354, 0x766a0abb,
94     0x81c2c92e, 0x92722c85,
95     0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3, 0xd192e819, 0xd6990624,
96     0xf40e3585, 0x106aa070,
97     0x19a4c116, 0x1e376c08, 0x2748774c, 0x3460bcb5, 0x391c0cb3, 0x4ed8aa4a,
98     0x5b9cca4f, 0x682e6ff3,
99     0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x90befffa, 0xa4506ceb,
100    0xbef9a3f7, 0xc67178f2
101 };
102
103 int main() {
104     std::string input;
105     std::cout << "Enter text to hash: ";
106     std::getline(std::cin, input);
107
108     std::string hashOutput = SHA256::hash(input);
109     std::cout << "SHA-256 Hash Digest: " << hashOutput << std::endl;
110
111     return 0;
112 }
```

Listing 2: SHA-256 Algorithm Implementation in C++

2.3 Sample Output

```
1 Enter text to hash: hello
2 SHA-256 Hash Digest:
   2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e7304336293869824
```

2.4 Result & Conclusion

The SHA-256 cryptographic hashing algorithm was implemented in C++. The program successfully processes user string input, applies message padding, executes the 64-round transformation compression, and yields a fixed 256-bit digest.

3 Assignment 3: Implementation and Verification of RSA Digital Signature Scheme

Aim / Objective: Implementation and verification of the RSA Digital Signature scheme in Python[cite: 5].

3.1 Theory & Overview

The RSA Digital Signature scheme relies on the asymmetry of asymmetric key cryptography[cite: 5]. A sender hashes a message using SHA-256 and encrypts the hash digest with their private key d to form a signature $S = H(M)^d \pmod{n}$ [cite: 5]. The recipient decrypts the signature using the sender's public key e ($H' = S^e \pmod{n}$)[cite: 5] and checks whether it matches the newly computed hash of the received message[cite: 5].

3.2 Python Code

```

1 import hashlib
2
3 def extended_gcd(a: int, b: int):
4     if b == 0:
5         return a, 1, 0
6     gcd, x1, y1 = extended_gcd(b, a % b)
7     return gcd, y1, x1 - (a // b) * y1
8
9 def mod_inverse(e: int, phi: int) -> int:
10    gcd, x, y = extended_gcd(e, phi)
11    if gcd != 1:
12        raise ValueError("Modular inverse does not exist.")
13    return (x % phi + phi) % phi
14
15 def generate_rsa_keys(p: int, q: int):
16    n = p * q
17    phi = (p - 1) * (q - 1)
18    e = 17 # Common coprime candidate
19    d = mod_inverse(e, phi)
20    return (e, n), (d, n)
21
22 def hash_message(message: str) -> int:
23    # Compute SHA-256 hash integer representation
24    digest = hashlib.sha256(message.encode()).hexdigest()
25    return int(digest, 16)
26
27 def sign_message(message: str, private_key: tuple) -> int:
28    d, n = private_key
29    m_hash = hash_message(message) % n
30    signature = pow(m_hash, d, n)
31    return signature
32
33 def verify_signature(message: str, signature: int, public_key: tuple) -> bool:
34    e, n = public_key
35    m_hash = hash_message(message) % n
36    decrypted_hash = pow(signature, e, n)
37    return m_hash == decrypted_hash
38
39 if __name__ == "__main__":
40    # Key Generation using prime numbers p = 61 and q = 53
41    public_key, private_key = generate_rsa_keys(61, 53)
42
43    message = "CONFIDENTIAL_DATA"
44    signature = sign_message(message, private_key)
45
46    print("=== RSA Key Pair Generation ===")
47    print(f"Public Key (e, n) : {public_key}")
48    print(f"Private Key (d, n): {private_key}\n")

```

```
49
50     print("=== Digital Signature Generation & Verification ===")
51     print(f"Original Message : {message}")
52     print(f"Digital Signature: {signature}\n")
53
54     # Verification on valid data
55     is_valid = verify_signature(message, signature, public_key)
56     print(f"Verification (Original Message): {'SUCCESS' if is_valid else 'FAILED'}")
57
58     # Verification on tampered data
59     tampered_message = "CONFIDENTIAL_DATA"
60     is_tampered_valid = verify_signature(tampered_message, signature, public_key)
61     print(f"Verification (Tampered Message): {'SUCCESS' if is_tampered_valid else 'FAILED'}")
```

Listing 3: RSA Digital Signature Scheme Implementation in Python

3.3 Sample Output

=== RSA Key Pair Generation ===

Public Key (e, n) : (17, 3233)

Private Key (d, n): (2753, 3233)

=== Digital Signature Generation & Verification ===

Original Message : CONFIDENTIAL_DATA

Digital Signature: 89

Verification (Original Message): SUCCESS

Verification (Tampered Message): FAILED

3.4 Result & Conclusion

The RSA Digital Signature scheme was successfully implemented and verified in Python, demonstrating authenticity verification for unaltered messages and detection of unauthorized message modifications[cite: 5].

4 Assignment 4: Implementation of ElGamal Digital Signature Scheme

Aim / Objective: To implement the ElGamal Digital Signature Scheme in Python, covering all three stages: key generation, signature generation (r, s) , and signature verification $(v_1 = v_2)$.

4.1 Theory & Mathematical Formulation

The ElGamal Digital Signature Scheme is an asymmetric key cryptographic signature algorithm based on the difficulty of computing discrete logarithms.

1. Key Generation:

- Select a large prime p and a primitive root $g \pmod{p}$.
- Choose a secret private key x such that $1 \leq x \leq p - 2$.
- Compute the public key parameter $y = g^x \pmod{p}$.
- Public Key: (p, g, y) , Private Key: x .

2. Signature Generation:

For a message hash m :

- Select a random secret key k such that $\gcd(k, p - 1) = 1$.
- Compute $r = g^k \pmod{p}$.
- Compute $s = k^{-1} \cdot (m - x \cdot r) \pmod{p - 1}$.
- The signature is the pair (r, s) .

3. Verification:

- Check that $0 < r < p$ and $0 < s < p - 1$.
- Compute $v_1 = g^m \pmod{p}$.
- Compute $v_2 = (y^r \cdot r^s) \pmod{p}$.
- The signature is valid if and only if $v_1 = v_2$.

4.2 Python Code

```

1 import math
2
3 # Function to calculate modular inverse of 'a' mod 'm' using Extended Euclidean
  Algorithm
4 def mod_inverse(a, m):
5     m0, y, x = m, 0, 1
6     if m == 1:
7         return 0
8     while a > 1:

```

```

9         q = a // m
10        m, a = a % m, m
11        y, x = x - q * y, y
12    if x < 0:
13        x += m0
14    return x
15
16 def main():
17     # 1. KEY GENERATION
18     p = 23 # Prime number
19     g = 5  # Primitive root (generator)
20     x = 6  # Private key (1 <= x <= p-2)
21
22     # Compute Public Key: y = (g^x) % p
23     y = pow(g, x, p)
24
25     print("=== 1. KEY GENERATION ===")
26     print(f"Public Key (p, g, y) : ({p}, {g}, {y})")
27     print(f"Private Key (x)      : {x}\n")
28
29     # 2. SIGNATURE GENERATION
30     m = 14 # Message hash (0 <= m <= p-1)
31     k = 15 # Random secret key such that gcd(k, p-1) = 1
32
33     # Check gcd condition
34     if math.gcd(k, p - 1) != 1:
35         print("Invalid k! gcd(k, p-1) must be 1.")
36         return
37
38     # Compute r = (g^k) % p
39     r = pow(g, k, p)
40
41     # Compute k^(-1) mod (p-1)
42     k_inv = mod_inverse(k, p - 1)
43
44     # Compute s = k^(-1) * (m - x * r) mod (p-1)
45     temp = (m - (x * r)) % (p - 1)
46     s = (k_inv * temp) % (p - 1)
47
48     print("=== 2. SIGNATURE GENERATION ===")
49     print(f"Message Hash (m)      : {m}")
50     print(f"Random Key (k)         : {k}")
51     print(f"Digital Signature       : (r = {r}, s = {s})\n")
52
53     # 3. SIGNATURE VERIFICATION
54     print("=== 3. SIGNATURE VERIFICATION ===")
55
56     # Range check: 0 < r < p and 0 < s < p-1
57     if not (0 < r < p) or not (0 < s < p - 1):
58         print("Signature Invalid: Range check failed!")
59         return
60
61     # Compute v1 = (g^m) % p
62     v1 = pow(g, m, p)
63
64     # Compute v2 = (y^r * r^s) % p
65     yr = pow(y, r, p)
66     rs = pow(r, s, p)
67     v2 = (yr * rs) % p
68
69     print(f"v1 (g^m mod p)          : {v1}")
70     print(f"v2 (y^r * r^s mod p)      : {v2}")
71
72     if v1 == v2:
73         print("\nRESULT: Signature is VALID!")
74     else:
75         print("\nRESULT: Signature is INVALID!")
76
77 if __name__ == "__main__":
78     main()

```

Listing 4: ElGamal Digital Signature Scheme Implementation in Python

4.3 Sample Output

=== 1. KEY GENERATION ===

Public Key (p, g, y) : (23, 5, 8)

Private Key (x) : 6

=== 2. SIGNATURE GENERATION ===

Message Hash (m) : 14

Random Key (k) : 15

Digital Signature : (r = 19, s = 8)

=== 3. SIGNATURE VERIFICATION ===

v1 ($g^m \bmod p$) : 13

v2 ($y^r * r^s \bmod p$) : 13

RESULT: Signature is VALID!

4.4 Observation & Result

- **Key Generation:** Public key components created: prime $p = 23$, generator $g = 5$, and computed $y = g^x \pmod{p} = 5^6 \pmod{23} = 8$. Private key $x = 6$ remains secret.
- **Signature Generation:** Random integer $k = 15$ satisfies $\gcd(15, 22) = 1$. $r = 5^{15} \pmod{23} = 19$. Modular inverse $k^{-1} \pmod{22} = 3$. Signature component $s = 3 \times (14 - 6 \times 19) \pmod{22} = 8$.
- **Verification:** Range check condition $0 < 19 < 23$ and $0 < 8 < 22$ is satisfied. Computed verification values yield $v_1 = 13$ and $v_2 = 13$. Since $v_1 = v_2$, the signature is successfully verified as **VALID**.